

Leena.app v401 - Backend Kurulum Ozeti

1. PostgreSQL Veritabani Kurulumu

```
psql postgres
CREATE DATABASE leena_v401;
\q

psql leena_v401 < leena_v401_schema_v2.sql
```

2. Proje Yapisi ve Klasorler

```
mkdir leena-v401-backend
cd leena-v401-backend
touch .env
mkdir utils
mkdir routes
```

3. .env Dosyasi

```
PGUSER=postgres
PGHOST=localhost
PGDATABASE=leena_v401
PGPASSWORD=
PGPORT=5432
JWT_SECRET=supersecureleena
```

4. db.js (utils/db.js)

```
const { Pool } = require('pg');
require('dotenv').config();

const pool = new Pool({
  user: process.env.PGUSER,
  host: process.env.PGHOST,
  database: process.env.PGDATABASE,
  password: process.env.PGPASSWORD,
  port: process.env.PGPORT,
});

module.exports = pool;
```

5. index.js

```
const express = require('express');
const cors = require('cors');
const pool = require('./utils/db');
const authRouter = require('./routes/auth');

const app = express();

app.use(cors());
```

```

app.use(express.json());

app.get('/', (req, res) => {
  res.send('Leena.app v401 API is running');
});

app.get('/api/organizers', async (req, res) => {
  try {
    const result = await pool.query('SELECT * FROM organizers ORDER BY id');
    res.json(result.rows);
  } catch (err) {
    console.error('Error fetching organizers:', err);
    res.status(500).json({ error: 'Internal server error' });
  }
});

app.post('/api/organizers', async (req, res) => {
  try {
    const { name, email, logo_url } = req.body;
    const result = await pool.query(
      'INSERT INTO organizers (name, email, logo_url) VALUES ($1, $2, $3) RETURNING *',
      [name, email, logo_url]
    );
    res.json(result.rows[0]);
  } catch (err) {
    console.error(err.message);
    res.status(500).send('Server error');
  }
});

app.use('/api/auth', authRouter);

app.listen(3000, () => {
  console.log(' Leena.app v401 backend running on http://localhost:3000');
});

```

6. auth.js (routes/auth.js)

```

const express = require('express');
const jwt = require('jsonwebtoken');
const bcrypt = require('bcrypt');
const router = express.Router();
const pool = require('../utils/db');
require('dotenv').config();

router.post('/login', async (req, res) => {
  const { email, password } = req.body;

  try {
    const result = await pool.query('SELECT * FROM organizers WHERE email = $1', [email]);

    if (result.rows.length === 0) {
      return res.status(401).json({ error: 'Invalid email or password' });
    }
  }

```

```
const organizer = result.rows[0];
const match = await bcrypt.compare(password, organizer.password_hash);

if (!match) {
  return res.status(401).json({ error: 'Invalid email or password' });
}

if (!process.env.JWT_SECRET) {
  throw new Error('JWT_SECRET not defined in .env');
}

const token = jwt.sign(
  {
    organizer_id: organizer.id,
    email: organizer.email,
  },
  process.env.JWT_SECRET,
  { expiresIn: '8h' }
);

res.json({ token });
} catch (err) {
  console.error('Login error:', err);
  res.status(500).json({ error: 'Internal server error' });
}
});

module.exports = router;
```